A PROJECT REPORT ON

HACKAGON

A Web Platform for Hosting and Joining Online Coding Competitions

INDIVIDUAL CONTRIBUTION REPORT

Member 4: Proctoring and Duels

Submitted in partial fulfilment of the requirements for the degree of

Bachelor of Technology in Computer Science and Engineering

Submitted by

Arjun Suman

(Member 4 of the HackAgon team: Hiten Sharma, Harsh Sharma, Hemanth Singh Tanwar, Arjun Suman, Akshat Rathore)

Under the guidance of

Er. Ram Babu Buri

Department of Computer Science and Engineering

Arya College of Engineering and IT, Jaipur

Session 2026-27

CERTIFICATE

This is to certify that the individual contribution report entitled “HackAgon: Individual Report of Member 4 (Proctoring and Duels)”, submitted by Arjun Suman, a student of the third year of Bachelor of Technology in Computer Science and Engineering at Arya College of Engineering and IT, Jaipur, is a record of his own work carried out during the session 2026-27 as a member of the HackAgon project team, under my guidance and supervision.

The work embodied in this report has not been submitted elsewhere for the award of any degree or diploma. The report is, in my opinion, worthy of consideration as part of the project submission.

Er. Ram Babu Buri	**Dr. Akhil Pandey**
Project Guide	Head of Department

Date: _____

Place: Jaipur

DECLARATION

I, Arjun Suman, hereby declare that the individual contribution report entitled “HackAgon: Individual Report of Member 4 (Proctoring and Duels)” is an original account of the work done by me as Member 4 of the HackAgon project team, under the guidance of the project guide named in the certificate.

The report is based on my own work, supported by the HackAgon Daily Work Log of the team (6 July 2026 to 6 October 2026) and the HackAgon Team Modules and 12-Week Plan. It has not been copied from any other source. Wherever the work of other team members is described, it is attributed to them by name or by member number.

Arjun Suman

Date: _____

Place: Jaipur

ACKNOWLEDGEMENT

I am grateful to everyone who stood beside me while I built my part of HackAgon. I thank our project guide for clear direction, patient reviews and useful suggestions from the first week to the last. I thank the Head of the Department of Computer Science and Engineering and the faculty of Arya College of Engineering and IT, Jaipur, for the learning environment and the laboratory facilities.

I thank my teammates Hiten Sharma, Harsh Sharma, Hemanth Singh Tanwar and Akshat Rathore. A lock is only worth something if the match page respects it, a flag is only worth something if a host can act on it, and a screen is only worth something if a player can understand it. Their work on accounts and rooms, on the judge and admin tools, on the database, money and deployment, and on the design and learning screens made my modules useful. Their reviews of my pull requests, and mine of theirs, taught me as much about teamwork as about code.

I also thank the open-source community. HackAgon stands on Java, Spring Boot, MySQL, Render, GitHub and many small libraries, all free to use and well documented, and on the browser features that made tab, fullscreen and camera checks possible. Finally, I thank my friends, who joined the mock matches and put up with my locked screens, and my family for their encouragement.

Arjun Suman

ABSTRACT

HackAgon is a web application for hosting and joining online coding competitions, with solo, duo, squad and duel rooms, paid tournaments with a funded prize pool, automatic judging, proctoring and an administrator back office. It is built by a team of five students. This report describes the contribution of Member 4, Arjun Suman, who owned two modules: Module 7, Proctoring and host dashboard, and Module 8, Duels and teams.

For Module 7, the report covers the browser spike on tab, minimize, fullscreen and camera events, the proctor API contract, the snapshot, violation and lock-state endpoints, the camera permission and face check, snapshots every 20 to 30 seconds, the lock held on the server, the 30 second network grace period, the locked screen, the host dashboard with its snapshot viewer and eliminate button, the 30 day snapshot deletion job and the 10-person proctoring trial. For Module 8, it covers the duel and team tables, the duel invite and waiting-queue APIs, duo and squad teams with size rules, team scoring in which each question counts once, automatic duel pairing, the platform daily room and practice duels, rematch, duel history and the link from duel results to achievements and mistakes. It then describes weeks 10 to 14: edge-case tests, three mock matches, the host guide, launch-week monitoring and the tuning that followed the first real event.

A full chapter is devoted to collaboration. I worked with Member 1 on the lock, consent and elimination, with Member 2 on cheating evidence and disputes, with Member 3 on tables, storage, load and deployment, and with Member 5 on the consent, camera and lock screens and on progress screens. In total I logged 194.0 hours over 67 working days, from 6 July to 6 October 2026. Every feature in the report is traced to a dated entry in the daily work log.

Keywords: proctoring, tab lock, webcam snapshots, host dashboard, duels, teams, false locks, grace period, collaboration, Java, Spring Boot, MySQL.

CONTENTS

1. INTRODUCTION..... 1
1.1 About the HackAgon Project..... 1
1.2 Purpose of This Report..... 1
1.3 Member 4 at a Glance..... 1
1.4 Sources and Method..... 2
1.5 Organisation of the Report..... 2
2. ROLE AND RESPONSIBILITIES..... 3
2.1 The Two Modules..... 3
2.2 Files, Pages and Responsibilities..... 3
2.3 Decisions Owned by Member 4..... 3
2.4 What Belonged to Other Members..... 4
2.5 Effort Summary..... 4
3. REQUIREMENTS AND DESIGN..... 7
3.1 Requirements Covered..... 7
3.2 Non-Functional Requirements That Shaped the Work..... 8
3.3 Position in the Architecture..... 8
3.4 Data Design..... 9
3.5 Design Principles..... 9
4. MODULE 7: PROCTORING AND HOST DASHBOARD..... 10
4.1 The Browser Spike (Week 1)..... 10
4.2 The Proctor API Contract (10 July)..... 10
4.3 The Three Endpoints (13 to 15 July)..... 10
4.4 Where Snapshots Are Stored (16 July)..... 10
4.5 Camera Permission and Face Check (10 August)..... 11
4.6 Snapshots Every 20 to 30 Seconds (11 August)..... 11
4.7 The Tab and Minimize Lock on the Server (12 August)..... 11
4.8 The 30 Second Network Grace Period (13 August)..... 11
4.9 The Locked Screen (14 August)..... 11
4.10 The Host Dashboard (17 to 20 August)..... 12
4.11 The Snapshot Retention Job (21 August)..... 13
4.12 The Real Proctoring Trial (24 to 28 August)..... 13
5. MODULE 8: DUELS AND TEAMS..... 14
5.1 Duel and Team Tables (17 July)..... 14
5.2 Duel Invite API and Waiting Queue (20 and 21 July)..... 14
5.3 Teams: Create, Join and Size Rules (22 to 24 July)..... 14
5.4 Screens: Duel Page, Team Join and Teammate List (27 to 29 July)..... 14
5.5 Testing the Duel Invite Flow (30 and 31 July)..... 14
5.6 Team Score Counts Each Question Once (3 August)..... 14
5.7 Automatic Duel Pairing (4 August)..... 14
5.8 The Daily Room and Practice Duels (5 and 6 August)..... 14
5.9 Testing Duo, Squad and Duel Play (7 August)..... 15
5.10 Rematch and Duel History (31 August and 1 September)..... 15
5.11 Duel Results into Achievements and Mistakes (2 and 3 September)..... 15
6. HARDENING, MOCK MATCHES AND LAUNCH..... 16
6.1 Edge-case Tests (7 to 11 September)..... 16
6.2 Mock Matches with Friends (14 to 17 September)..... 16
6.3 The Host Guide (18 September, updated 1 October)..... 16
6.4 Code Freeze and Launch Week (21 to 25 September)..... 16
6.5 After Launch (28 September to 2 October)..... 17

6.6 Final Review and Submission (5 and 6 October)....17
7. COLLABORATION WITH OTHER MEMBERS....18
7.1 Overview of Dependencies and Joint Functions....18
7.2 Collaboration with Member 1 (Hiten Sharma)....19
7.3 Collaboration with Member 2 (Harsh Sharma)....19
7.4 Collaboration with Member 3 (Hemanth Singh Tanwar)....19
7.5 Collaboration with Member 5 (Akshat Rathore)....20
7.6 Team-Wide Activities....20
7.7 Summary of Collaboration....20
8. TESTING AND VERIFICATION....22
8.1 Test Strategy....22
8.2 Tests Recorded in the Work Log....22
8.3 Bug-fix Days....22
8.4 Verification by Other Members....22
8.5 Points to Note....23
9. WEEK-BY-WEEK SUMMARY....24
9.1 Weeks 1 and 2: Spike and Endpoints....25
9.2 Weeks 3 to 5: Duels, Teams and Formats....25
9.3 Weeks 6 to 9: Lock, Dashboard, Trial and Duel Polish....25
9.4 Weeks 10 to 12: Edge Cases, Mock Matches and Launch....26
9.5 Weeks 13 and 14: After Launch and Handover....26
10. CHALLENGES, DECISIONS AND LIMITATIONS....27
10.1 Challenges and How They Were Met....27
10.2 Delivered Features....27
10.3 Notes on the Sources....28
10.4 Limitations....28
11. CONCLUSION AND FUTURE SCOPE....29
11.1 Conclusion....29
11.2 Lessons Learnt....29
11.3 Future Scope....29
REFERENCES....30
APPENDIX A: COMPLETE DAILY WORK LOG OF MEMBER 4....31
APPENDIX B: DATED JOINT WORK WITH OTHER MEMBERS....34

LIST OF FIGURES

Figure 2.1: Hours logged by Member 4 per kind of work.....5
Figure 2.2: Timeline of Member 4’s work across the fourteen logged weeks.....5
Figure 2.3: Share of logged hours by project stage.....6
Figure 3.1: Layered architecture around Member 4’s modules.....8
Figure 4.1: How a browser event or a network drop leads to a lock, a grace period and the host dashboard.....12
Figure 4.2: Sequence diagram of snapshots, violations, lock state and the dashboard.....12
Figure 5.1: Duel lifecycle: invite or queue, pairing, play, result, rematch, history and progress…15
Figure 7.1: Collaboration map of Member 4 with the other four members.....18
Figure 7.2: Joint work items per partner (from Appendix B).....21
Figure 9.1: Cumulative hours logged by Member 4 week by week.....25
Figure 9.2: Hours logged per day, 6 July to 6 October 2026 (one bar per weekday).....25

LIST OF TABLES

Table 1.1: Summary of Member 4 1
Table 2.1: Files and pages of Member 4 3
Table 2.2: Hours by area of work 4
Table 3.1: Requirements and Member 4’s contribution 7
Table 3.2: Data used by Member 4’s modules 9
Table 4.1: Week 1 activities of Member 4 10
Table 6.1: Edge cases tested in week 10 16
Table 6.2: Work of weeks 10 to 14 17
Table 7.1: Matrix of joint functions 18
Table 8.1: Tests carried out by Member 4 22
Table 9.1: Weekly summary of Member 4’s work 24
Table 10.1: Challenges and responses 27
Table 10.2: Status of Member 4’s features 27
Table A.1: Daily work log of Member 4 31
Table B.1: Dated joint work 34

1. INTRODUCTION

1.1 About the HackAgon Project

HackAgon is a web platform on which anyone can host or join an online coding competition. A college club, a class or a group of friends opens a room, picks questions, invites players and starts the match. When the stakes are higher, the host can run a paid tournament: the prize pool is funded in advance, and after a flat 1% platform fee it is paid out by a published split. The product is built with Java and Spring Boot, a MySQL database, plain HTML pages and a set of shared browser scripts and style files.

Five students of Arya College of Engineering and IT, Jaipur, built it together. The work was split into ten modules, and each member carries modules of the same total weight. This report belongs to **Member 4, Arjun Suman**, whose area in the team plan is **Proctoring and Duels**.

1.2 Purpose of This Report

The team plan says what each module should deliver and when. The daily work log says what each member really did on each day. Neither document tells the story of one member from start to finish. This report does that for me: what I built, in which week, how I tested it and which of my teammates' modules it touched.

A contest platform is only as good as its fairness. When a match is played for money, every player must be sure that nobody is cheating and that nobody honest is thrown out by mistake. My two modules deal with exactly that, together with the head-to-head and team formats that make matches more social. Wherever my work meets the work of another member, such as the lock that Member 1's match page enforces, the similarity flags of Member 2, the tables of Member 3 or the screens of Member 5, I name the shared function, the interface and the dates on which each side did its part.

1.3 Member 4 at a Glance

Table 1.1: Summary of Member 4

Item	Details
Name	Arjun Suman (Member 4)
Role in the team plan	Proctoring and Duels
Modules owned	Module 7: Proctoring and host dashboard; Module 8: Duels and teams
Main files	ProctorController.java, the proctoring script of the match page, the duel part of PlayController.java (shared), duel and team tables (added with Member 3)
Main pages	host-dashboard, duel, team join screen, locked screen (designed by Member 5), teammate list inside a room
Period covered by the log	6 July 2026 to 6 October 2026 (weekdays), project weeks 1 to 14
Work logged	67 weekdays, all marked Completed; **194.0 hours** (2 to 4 hours a day; 14.5 hours in each of weeks 1 to 13 and 5.5 hours in the two-day week 14)
Collaborated with	Member 1 (Hiten Sharma), Member 2 (Harsh Sharma), Member 3 (Hemanth Singh Tanwar), Member 5 (Akshat Rathore)

1.4 Sources and Method

I wrote this report from my own work and checked it against two team documents: the HackAgon Team Modules and 12-Week Plan, and the HackAgon Daily Work Log. The plan gives the module descriptions, owners, file names, weekly tasks and risks. The log gives the day-by-day record of all five members, including my 67 entries and the entries of the other four members that touch my work. Dates are written as in the log.

The plan is my guide to what each task was supposed to deliver. The log is my guide to what was done and on which day, so every date in this report comes from the log. Two differences between the documents should be stated openly. First, the plan describes twelve weeks and proposes a calendar that begins on 12 October 2026, whereas the log covers fourteen project weeks from 6 July to 6 October 2026; I follow the log for dates and use the plan only for the meaning and ownership of tasks. Second, the logs of different members place launch-week events on slightly different days (Section 6.4); I follow each log for its own entries and say so where it matters.

1.5 Organisation of the Report

Chapter 2 explains my role, my files and my effort. Chapter 3 sets out the requirements and the design. Chapter 4 describes Module 7, proctoring and the host dashboard, and Chapter 5 describes Module 8, duels and teams. Chapter 6 covers weeks 10 to 14: edge-case tests, mock matches, the host guide, launch and the weeks after it. Chapter 7 is about collaboration with the other four members. Chapter 8 reports testing, Chapter 9 gives a week-by-week account, and Chapter 10 discusses challenges, decisions and limitations. Chapter 11 closes with lessons and future scope. Appendix A reproduces my daily log, and Appendix B lists the dated joint work with the other members.

2. ROLE AND RESPONSIBILITIES

2.1 The Two Modules

In the team plan every member owns one larger and one smaller module, so that the load is equal. I own the two modules that keep a contest honest and that give it more formats than a solo race.

Module 7: Proctoring and Host Dashboard (weight 3)

This module covers tournament mode: the webcam face check and snapshots, the tab and minimize lock, the lock state kept on the server, the violation log, the host dashboard, elimination from the host side and the similarity check that Member 2 feeds into the dashboard. Its main files are ProctorController.java, the host-dashboard page and the proctoring script of the match page.

Module 8: Duels and Teams (weight 2)

This module covers duel invites and the waiting queue, duo and squad teams, team scoring, and the built-in practice duels and daily room. Its main files are the duel page, the duel part of PlayController.java and the team tables. The weights 3 and 2 give me a load of 5 points, the same as every other member.

2.2 Files, Pages and Responsibilities

Table 2.1: Files and pages of Member 4

File or page	Responsibility
ProctorController.java	Snapshot upload, violation log and lock-state endpoints, written to the proctor API contract of 10 July
Proctoring script of the match page	Camera permission and face check before the start, snapshot every 20 to 30 seconds, reporting of tab, minimize and fullscreen events
host-dashboard page	Live player list, camera status, violation counts, snapshot viewer and eliminate button
Snapshot deletion job	Removes snapshots after 30 days
PlayController.java (duel part, shared)	Duel invite, waiting queue, automatic pairing, team rules and team scoring; the submission endpoint in the same file belongs to Member 1
duel page and team join screen	Connected by me to the duel and team APIs; styled with the components of Member 5
Duel and team tables	Added with Member 3 on 17 July; the schema itself is owned by Member 3
Locked screen	Designed by Member 5 on 12 August, built and tested by me on 14 August

2.3 Decisions Owned by Member 4

- **The server holds the lock.** The browser only reports what it sees. Whether a player is locked is decided and stored on the server, so closing a tab or reloading a page cannot clear it.

- **Honest players come first.** A short network drop is not cheating. The 30 second grace period exists so that a weak connection does not cost a player the match.
- **Evidence, not verdicts.** Snapshots, camera status and violation counts are shown to the host, and the host decides whether to eliminate a player.
- **Keep personal images only as long as needed.** Snapshots are deleted by a scheduled job after 30 days.
- **A team question counts once.** However many teammates solve it, a question adds to the team score a single time.
- **Always something to play.** The platform daily room and the always-open practice duels mean a player can find a match without waiting for a host.

2.4 What Belonged to Other Members

I did not own the accounts, rooms, results or the rule that a locked player is refused on every request (Member 1), the judge, question bank, admin screens, disputes or the similarity comparison itself (Member 2), the money tables, schema, deployment and load tests (Member 3), or the design system, the consent, camera check and lock screen designs, achievements and My mistakes (Member 5). My code meets these parts at agreed points, described in Chapter 7.

2.5 Effort Summary

I logged work on all 67 weekdays between 6 July and 6 October 2026, a total of **194.0 hours**. Daily hours ranged from 2 to 4, each full week came to 14.5 hours, and the last week (5 and 6 October) came to 5.5 hours. Every entry is marked Completed. Table 2.2 groups the hours by area of work; each of the first twelve areas is exactly one project week.

Table 2.2: Hours by area of work

Area of work	Weeks	Days	Hours	Share
Browser spike and proctor API contract	1	5	14.5	7.5%
Proctor endpoints and team tables	2	5	14.5	7.5%
Duel and team APIs	3	5	14.5	7.5%
Duel and team screens	4	5	14.5	7.5%
Team scoring, pairing and practice duels	5	5	14.5	7.5%
Webcam and tab lock	6	5	14.5	7.5%
Host dashboard	7	5	14.5	7.5%
Real proctoring trial	8	5	14.5	7.5%
Duel polish	9	5	14.5	7.5%
Proctoring edge cases	10	5	14.5	7.5%
Mock matches and host guide	11	5	14.5	7.5%
Launch watch	12	5	14.5	7.5%
Post-launch fixes and handover	13 to 14	7	20.0	10.3%
Total	**1 to 14**	**67**	**194.0**	**100%**

Each log entry falls into one of six kinds of work. Building new features took almost half of the logged time. Testing took about a fifth, and the rest was shared by monitoring and review, fixing and tuning, research and design decisions, and documentation. Figure 2.1 shows the split.

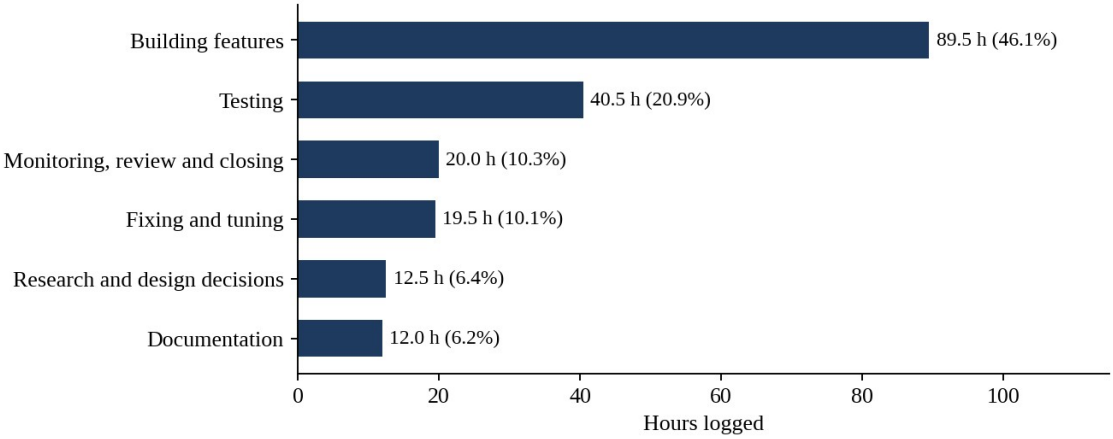


Figure 2.1: Hours logged by Member 4 per kind of work

Figure 2.2 shows when each area was worked on. The first two weeks went to the browser spike and the proctor endpoints, weeks 3 to 5 to duels and teams, and weeks 6 to 8 to the real proctoring features and their first trial. Week 9 went back to duels, and weeks 10 to 12 were the testing and launch weeks.

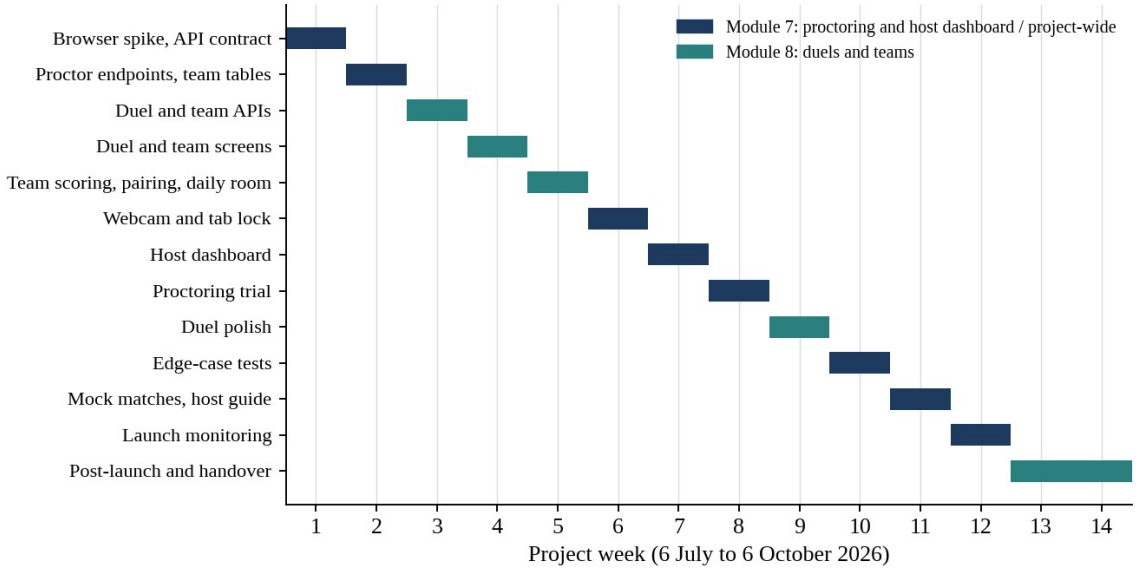


Figure 2.2: Timeline of Member 4’s work across the fourteen logged weeks

Figure 2.3 groups the same hours into five stages. The largest stage is weeks 6 to 9, because the webcam and tab lock, the host dashboard, the trial and the duel polish all fall there.

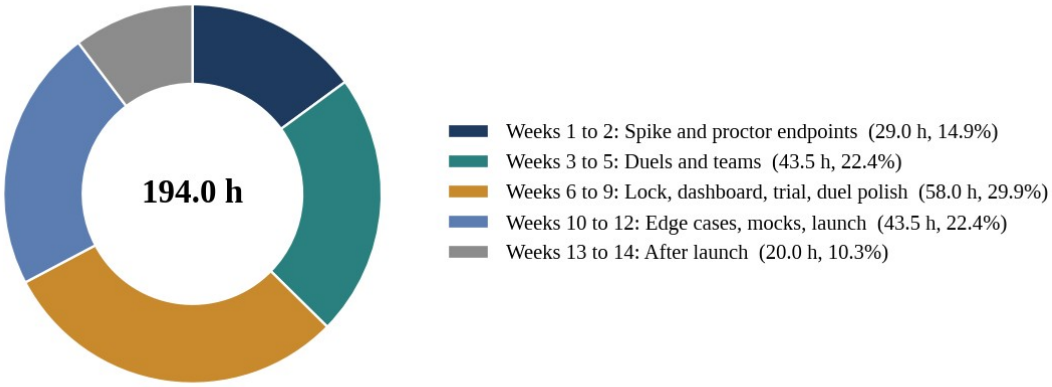


Figure 2.3: Share of logged hours by project stage

3. REQUIREMENTS AND DESIGN

3.1 Requirements Covered

The plan states what each module must deliver. Table 3.1 lists the requirements that my tasks address, with the dates on which the log shows the work.

Table 3.1: Requirements and Member 4’s contribution

Requirement (from the plan)	Contribution and date
Test tab-change, minimize, fullscreen-exit and camera capture in Chrome, Edge and Firefox	Tested and browser differences noted (6 to 9 July)
Proctor API contract for snapshot, violation and lock state	Written (10 July)
Snapshot upload, violation log and lock-state endpoints	Built (13 to 15 July)
Decide where snapshots are stored	Decided (16 July)
Duel and team tables in the schema	Added with Member 3 (17 July)
Duel invite and waiting-queue API	Built (20 and 21 July)
Create team and join team for duo and squad; team size rules	Built and tested (22 to 24 July)
Duel page connected; team join screen; teammate list inside a room	Built (27 to 29 July)
Team score counts each question once	Built (3 August)
Automatic duel pairing	Built (4 August)
Platform daily room and always-open practice duels	Built (5 and 6 August)
Camera permission and face check before start	Added (10 August)
Snapshot every 20 to 30 seconds	Added (11 August)
Tab, minimize and fullscreen events set the lock on the server	Added (12 August)
30 second network grace period; locked screen	Added and tested (13 and 14 August)
Host dashboard with live player list, camera status, violation counts, snapshot viewer and eliminate button	Built (17 to 20 August)
Job that deletes snapshots after 30 days	Built (21 August)
10-person proctoring trial: count false locks, tune grace time, fix browser problems	Run and tuned (24 to 28 August); fixes on 4 September
Rematch and duel history; duel results feed achievements and mistakes	Built (31 August to 3 September)
Edge cases: refresh, re-login, two tabs, unplugged camera, slow network; locks survive and honest players are not punished	Tested (7 to 11 September)
Three mock matches with friends; short host guide for tournament mode	Run and written (14 to 18 September)

Requirement (from the plan)	Contribution and date
Monitor locks and snapshots during the first real event; fix urgent problems	Done (21 to 24 September)

3.2 Non-Functional Requirements That Shaped the Work

- **Fairness without false accusations.** A lock must stop real cheating, yet an honest player with a poor connection must not be punished. Much of weeks 6, 8 and 10 went into this balance.
- **Server authority.** Locks, violations and snapshots are stored on the server, and the browser is only a reporter.
- **Privacy.** The camera is used only in tournament mode, after the consent page that Member 1 built, and snapshots are removed after 30 days.
- **Browser variety.** Players use different browsers, so the spike and the trial covered Chrome, Edge and Firefox.
- **Resilience.** Refreshing, logging in again, opening a second tab, unplugging the camera or a slow network must not break a match or wrongly lock a player.
- **Honest numbers.** As the team rule states, no demo data is placed in the app; duel history and counters come from real matches.

3.3 Position in the Architecture

The application is a Java and Spring Boot server with MySQL. Pages talk to the server through js/api.js, which Member 5 maintains. My code sits in the controller package (ProctorController.java and the duel part of PlayController.java) and uses Member 3's repository layer. Figure 3.1 shows the layers and where my code sits.

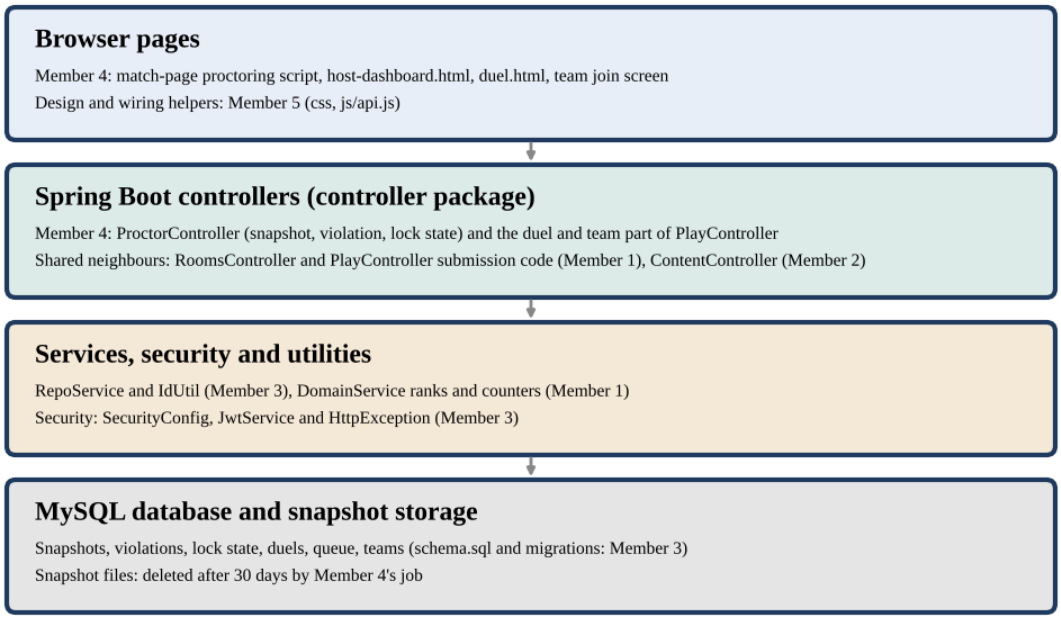


Figure 3.1: Layered architecture around Member 4’s modules

3.4 Data Design

On 17 July I added the duel and team tables together with Member 3, who owns schema.sql, the migration runner and the repository layer. The exact table and column names are in schema.sql and are not repeated here. Table 3.2 describes the data my modules use.

Table 3.2: Data used by Member 4's modules

Data	Purpose
Snapshots	Image taken from the player's camera every 20 to 30 seconds in tournament mode; kept for 30 days
Violations	Record of each reported event (tab change, minimize, fullscreen exit and similar) per player and room; the counts are shown on the host dashboard
Lock state	Whether a player is currently locked in a room, set by the server and read by the match page
Duels	Invites, waiting-queue entries, pairings, results, rematch links and history
Teams	Team name, size rules for duo and squad, members and their room
Team score	Score of a team, in which each question counts once

3.5 Design Principles

- **Contract before code.** The proctor API contract of 10 July fixed the three endpoints before anyone depended on them.
- **Stubs first, real later.** The endpoints began as simple versions in week 2, and the real camera, snapshot and lock behaviour came in week 6.
- **Be slow to lock and quick to explain.** The grace period delays a lock caused by the network, and the locked screen tells the player what happened.
- **Test the unhappy path.** Week 10 was spent on refresh, two tabs, an unplugged camera and a slow network, because honest players meet these situations more often than cheaters do.
- **One small step, one check.** Most building weeks end with a test or a bug-fix day, as the team's definition of done asks.

4. MODULE 7: PROCTORING AND HOST DASHBOARD

4.1 The Browser Spike (Week 1)

I began with a spike instead of code for the product. Before promising that tab changes, minimizing and fullscreen exits can be caught, I wanted to see how real browsers behave. In a plain test page I tried the tab-change event in Chrome (6 July), the minimize and fullscreen-exit events (7 July) and camera capture in Edge and Firefox (8 July). On 9 July I wrote down the differences I had seen.

Table 4.1: Week 1 activities of Member 4

Date	Hours	Activity
06-07-2026	3	Tested tab-change event in Chrome
07-07-2026	2.5	Tested minimize and fullscreen-exit events
08-07-2026	4	Tested camera capture in Edge and Firefox
09-07-2026	3	Noted browser differences
10-07-2026	2	Wrote proctor API contract for snapshot, violation, lock state

The log does not list the individual differences, so I do not list them here either. Their effect is visible later: the trial of week 8 and the edge-case tests of week 10 both include fixes for problems on different browsers.

4.2 The Proctor API Contract (10 July)

The plan asks every pair of modules to agree the request and response of a shared API in a contract file before connecting. On 10 July I wrote the proctor API contract for three calls: uploading a snapshot, logging a violation and reading the lock state. Member 1 needed the lock state for the match page, and Member 5 needed to know what the lock screen had to show, so a written contract let all three of us work in parallel. Member 1 later brought api-contract.md up to date for accounts and rooms on 1 October, with the proctor contract as one of the references.

4.3 The Three Endpoints (13 to 15 July)

In week 2 I built the three endpoints in ProctorController.java, one a day. On 13 July I built the snapshot upload endpoint, on 14 July the violation log endpoint and on 15 July the lock-state endpoint. The plan describes this first version as stubs: they accept and store what the contract promises, so that other modules can call them while the real proctoring behaviour is still to come. The camera and the lock rules arrived in week 6 and filled in what the stubs had reserved.

4.4 Where Snapshots Are Stored (16 July)

Snapshots are images, and images do not belong in a database row. On 16 July I chose where they are stored. The plan warns that free hosting services can lose local files, and its fallback for that risk is to keep snapshots in external storage. My log records the decision but not the name of the storage, so I leave that detail out. What the decision made possible was the retention job of week 7 and the snapshot viewer on the dashboard.

4.5 Camera Permission and Face Check (10 August)

In week 6 the module became real. On 10 August I added the camera permission request and a face check that runs before a tournament match starts. The player is asked for the camera first, and the match is not started until the check passes. Member 5 designed the camera check screen on 11 August, and Member 1's consent page of 11 August comes before it, so a player always sees what will happen to the camera before it is switched on.

4.6 Snapshots Every 20 to 30 Seconds (11 August)

On 11 August I added the periodic snapshot: while a tournament match is running, the page sends a picture to the snapshot endpoint every 20 to 30 seconds. Using a range instead of one fixed interval makes the timing less predictable and spreads the load on the server and the network. The pictures are what the host sees later in the snapshot viewer.

4.7 The Tab and Minimize Lock on the Server (12 August)

On 12 August I made tab changes, minimizing and leaving fullscreen set a lock on the server. The browser reports the event to the violation log endpoint, and the server updates the lock state of that player in that room. Because the state is on the server, reloading the page or opening another tab does not remove it. This is the same day on which Member 1 made the match page refuse locked players, and on 13 August Member 1 tested that check on every request, so the two halves of the lock were finished and tested side by side.

4.8 The 30 Second Network Grace Period (13 August)

A lock that fires on every dropped connection would punish people for bad Wi-Fi. On 13 August I added a 30 second grace period for network problems. A player whose connection drops has a short time to return before the drop counts. After the trial of week 8 I tuned the grace time (26 August), and on 29 September I tuned the lock rules again to reduce false locks. The log does not record the final values, so I do not state them.

4.9 The Locked Screen (14 August)

The locked screen is what a locked player sees. Member 5 designed it on 12 August, and on 14 August I built it and tested it. A good lock screen matters for fairness as much as the lock itself: a player who is locked wrongly must at least understand what happened and be able to tell the host.

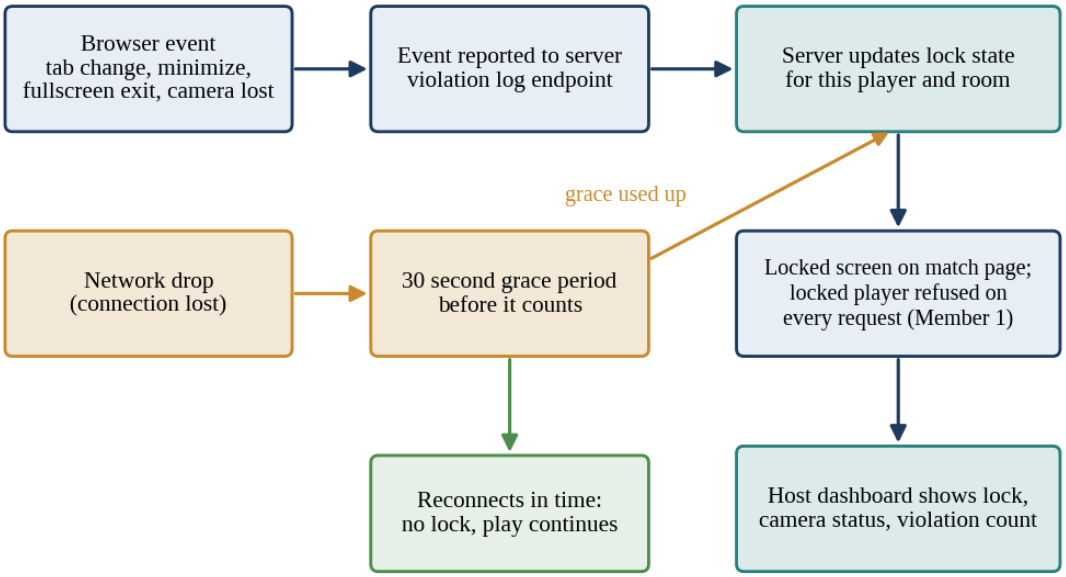


Figure 4.1: How a browser event or a network drop leads to a lock, a grace period and the host dashboard

Figure 4.2 shows the same behaviour as a sequence between the player's browser, the match-page script, the proctor routes, the storage and the host dashboard.

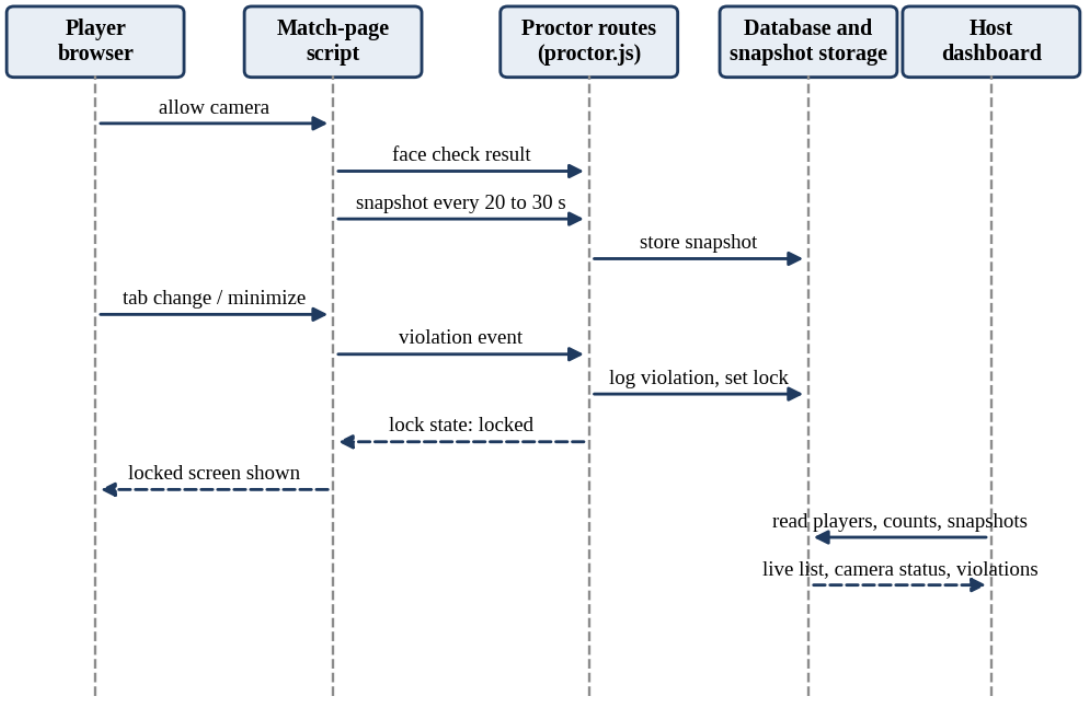


Figure 4.2: Sequence diagram of snapshots, violations, lock state and the dashboard

4.10 The Host Dashboard (17 to 20 August)

The host dashboard is where the host watches a tournament. On 17 August I built the live player list. On 18 August I added the camera status and the violation count of each player. On 19 August I built the snapshot viewer, and on 20 August I added the eliminate button. The button calls the elimination logic of Member 1, which asks for a reason, blocks the

player from rejoining and removes the player's score from the ranking (17 and 18 August). The dashboard therefore gives the host evidence in one place and one action that has a clear effect.

Member 2 produced the similarity flag list for the host dashboard on 12 August and showed it in the admin area on 13 August. My log does not record a separate step for showing that list on the dashboard, so I describe it as Member 2's output intended for the dashboard, and I make no further claim about it.

4.11 The Snapshot Retention Job (21 August)

On 21 August I built a job that deletes snapshots after 30 days. Pictures of players are personal data, and keeping them for longer than a dispute or review needs would be a risk without a benefit. I checked the job again on 2 October, near the end of the project.

4.12 The Real Proctoring Trial (24 to 28 August)

In week 8 I ran the first real trial: a 10-person test match (24 August). Afterwards I counted the false locks (25 August), tuned the grace time (26 August), fixed problems that appeared in different browsers (27 August) and wrote down the findings (28 August). On 4 September I fixed the remaining problems from the trial. The log does not give the number of false locks or the findings, so this report describes the method and the outcome (a tuned grace time and browser fixes) but not the numbers.

5. MODULE 8: DUELS AND TEAMS

5.1 Duel and Team Tables (17 July)

On 17 July I added the duel and team tables together with Member 3. The schema had been fixed in its first version on 13 July, and the migration runner and the repository layer followed on 14 and 15 July, so my tables were added through the same route as the others. Because shared files such as schema.sql are changed through Member 3, the work was done together instead of by me alone.

5.2 Duel Invite API and Waiting Queue (20 and 21 July)

A duel is a head-to-head match. On 20 July I built the duel invite API, which lets one player challenge another. On 21 July I built the waiting-queue API for a player who wants a duel but has no opponent yet. Together they cover both ways to start: invite a friend, or wait for a stranger.

5.3 Teams: Create, Join and Size Rules (22 to 24 July)

Duo and squad rooms need teams. I built the create-team API on 22 July and the join-team API for duo and squad on 23 July. On 24 July I added the team size rules and tested them. The rules stop a duo from having three members and a squad from having more than its limit.

5.4 Screens: Duel Page, Team Join and Teammate List (27 to 29 July)

I connected the existing duel.html to the duel API on 27 July and built the team join screen on 28 July. On 29 July I built the teammate list inside a room, so that players in a team match can see who is on their side. The pages use the components of Member 5, which were finished from 14 to 16 July, and the room page itself is the match page that Member 5 built from 27 to 30 July.

5.5 Testing the Duel Invite Flow (30 and 31 July)

On 30 July I tested the duel invite flow from invitation to start, and on 31 July I fixed the bugs that I found. The same two days were the final days before Milestone 1, which Member 3 deployed to staging on 30 July and checked on 31 July, so my duel and team code was in a clean state when the first playable build was tested.

5.6 Team Score Counts Each Question Once (3 August)

In a team match several teammates may solve the same question. On 3 August I made the team score count each question only once, so that a team cannot raise its score by solving the same question again. Member 1 added the final rank and tie-break rule on the same day and built the results page data on 4 August, so the team score and the rank rules had to agree for team rooms.

5.7 Automatic Duel Pairing (4 August)

On 4 August I built automatic duel pairing. When two players are waiting in the queue, the server pairs them and a duel room is made, so nobody has to arrange a match by hand.

5.8 The Daily Room and Practice Duels (5 and 6 August)

A new platform is quiet at first, and a quiet platform has nothing to offer a new visitor. On 5 August I built the platform daily room, and on 6 August the always-open practice duels. Both are run by the platform, so a player can always find a match.

5.9 Testing Duo, Squad and Duel Play (7 August)

On 7 August I tested duo, squad and duel play together. This was the end of the formats block, and it showed that the three formats work with the same room and match page.

5.10 Rematch and Duel History (31 August and 1 September)

In week 9 I made duels more complete. On 31 August I built the rematch, which lets the two players of a finished duel play again. On 1 September I built the duel history, a list of past duels for a player.

5.11 Duel Results into Achievements and Mistakes (2 and 3 September)

On 2 September I fed duel results into achievements, and on 3 September into the My mistakes list. Member 5 had built achievements with metrics from real counts, including duels won (17 to 20 August), and the My mistakes list (5 to 7 August), so I connected the duel side to screens that already existed. On 4 September I fixed the remaining problems from the proctoring trial.

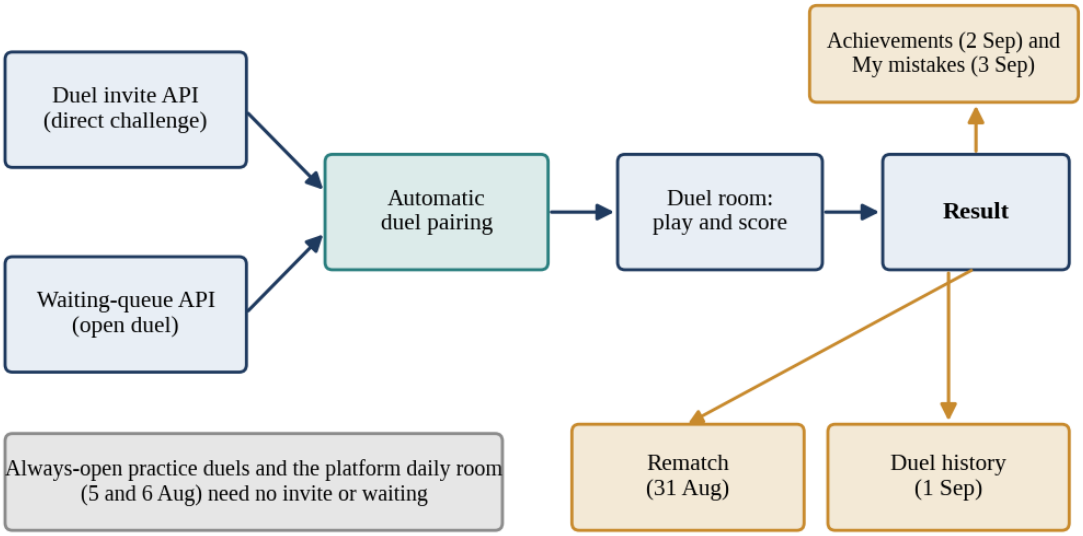


Figure 5.1: Duel lifecycle: invite or queue, pairing, play, result, rematch, history and progress

6. HARDENING, MOCK MATCHES AND LAUNCH

Weeks 10 to 14 turned a working feature set into a product that was ready for real players. This chapter covers my work in those weeks.

6.1 Edge-case Tests (7 to 11 September)

In week 10 I tested the situations that honest players meet and that a simple test page never shows. The plan calls them the edge cases of proctoring.

Table 6.1: Edge cases tested in week 10

Date	Situation tested	Why it matters
07-09-2026	Refresh and re-login during a match	A player who refreshes must not lose the lock state or be wrongly locked
08-09-2026	Two tabs open	A second tab is a tab change; the lock must treat it correctly
09-09-2026	Camera unplugged	A missing camera must show as a camera problem, not pass silently
10-09-2026	Slow network	The grace period must protect honest players on a weak connection
11-09-2026	Locks survive and honest players are not punished	The final confirmation of the whole week

In the same week Member 3 simulated 200 and then 500 players polling one room (7 and 8 September) and Member 2 ran the judge load test, so the app was being stressed from three sides.

6.2 Mock Matches with Friends (14 to 17 September)

Tests by one person find only what that person thinks of. In week 11 I ran three mock matches with friends, one on each of 14, 15 and 16 September, and on 17 September I fixed the bugs they exposed. During the same week Member 2 tested the whole app as a stranger would, from signup to match (14 September).

6.3 The Host Guide (18 September, updated 1 October)

A feature that a host cannot understand will not be used. On 18 September I wrote a short host guide for tournament mode, and on 1 October I updated it, after the first real event had shown what hosts and players actually needed. The guide matches the webcam consent text that Member 1 wrote on 17 September.

6.4 Code Freeze and Launch Week (21 to 25 September)

On 21 September I prepared the monitoring for the event. On 22 September my log records that I monitored locks during the event, and on 23 September that I monitored snapshots during the event. On 24 September I fixed urgent proctoring problems, and on 25 September the whole team held its retro.

The logs of the members differ by a day on launch events. Member 3’s log records the production deployment on 22 September and the small real tournament with friends on 23 September, while Member 1’s log records monitoring from 22 to 24 September. I follow

my own log for my entries and Member 3's log for the deployment and tournament dates, and I read my 22 September monitoring as the day the production service went live.

6.5 After Launch (28 September to 2 October)

On 28 September I reviewed the proctoring logs from the event. On 29 September I tuned the lock rules to reduce false locks, following that review of the event logs. On 30 September I re-tested duels and teams, and on 1 October I updated the host guide. On 2 October I checked the snapshot deletion job.

6.6 Final Review and Submission (5 and 6 October)

On 5 October I reviewed my teammates' pull requests, and on 6 October I did the final review and the submission preparation, which ended the logged work.

Table 6.2: Work of weeks 10 to 14

Week	Dates	Work of Member 4
10	07-09 to 11-09	Refresh, re-login, two tabs, unplugged camera and slow-network tests; confirmed locks survive
11	14-09 to 18-09	Three mock matches with friends; bugs fixed; short host guide for tournament mode
12	21-09 to 25-09	Monitoring prepared; locks and snapshots monitored during the event; urgent fixes; retro
13	28-09 to 02-10	Proctoring logs reviewed; lock rules tuned; duels and teams re-tested; host guide updated; deletion job checked
14	05-10 to 06-10	Pull request review; final review and submission preparation

7. COLLABORATION WITH OTHER MEMBERS

7.1 Overview of Dependencies and Joint Functions

The plan names the pairs that depend on each other, and for my modules the dependencies run in every direction. A lock that I set is useless unless the match page of Member 1 obeys it. A flag from Member 2 is useful only if a host can see it. My tables live inside the schema of Member 3, and my screens look and feel right because of the design system of Member 5. Figure 7.1 shows my collaboration map.

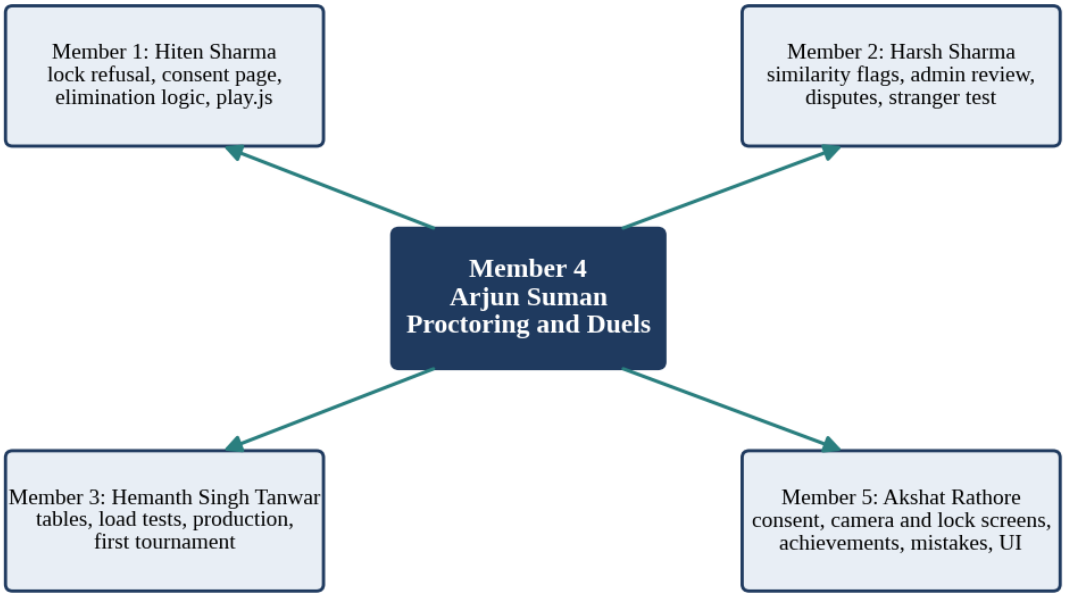


Figure 7.1: Collaboration map of Member 4 with the other four members

Table 7.1: Matrix of joint functions

Joint function	Member 4 side	Other side
Lock enforcement	Lock state on the server, grace period, locked screen	Member 1: match page refuses locked players on every request
Consent before camera	Camera permission, face check, snapshots	Member 1: consent page and webcam consent text; Member 5: consent and camera check screens
Elimination	Eliminate button on the host dashboard	Member 1: elimination logic with reason, no rejoin, removal from ranking
Cheating evidence	Dashboard with camera status, violation counts and snapshot viewer	Member 2: similarity flag list; review tools for flagged cheating and eliminated players
Disputes and false locks	Grace period, trial, tuning of lock rules	Member 2: dispute form, admin review, disputes of the first event
Team and duel rooms	Duel and team APIs, teammate list, team scoring	Member 1: join and start, ranks and tie-break; PlayController.java is shared

Joint function	Member 4 side	Other side
Tables and storage	Duel and team tables; snapshot storage and deletion job	Member 3: schema, migrations, repository layer, backups, production service
Load and edge cases	Refresh, two-tab, camera and slow-network tests	Member 3: load simulation of 200 and 500 players, scoreboard cache
Progress screens	Duel results fed to achievements and mistakes	Member 5: achievements with real counts, My mistakes list
Screens and look	Duel page, team join, locked screen wired up	Member 5: components, navigation, lock screen design
Launch	Monitoring of locks and snapshots	Members 1 and 3: signups and OTP; deployment, first tournament, payouts

7.2 Collaboration with Member 1 (Hiten Sharma)

Member 1 is my closest partner on the lock. I wrote the proctor contract on 10 July and built the lock-state endpoint on 15 July. On 12 August I set the tab and minimize lock on the server, and on the same day Member 1 made the match page refuse locked players; the check was tested on every request on 13 August, the day I added the grace period. The consent page that Member 1 built on 11 August comes before my camera check. On 17 and 18 August Member 1 built the elimination logic, with a reason, no rejoin and removal from the ranking, and on 20 August my eliminate button on the host dashboard began to call it.

The duel and team formats also rest on Member 1’s rooms. Rooms are created, joined and started through the logic that Member 1 built on 27 and 28 July, and PlayController.java is a file that both of us work in. My team scoring rule of 3 August was built on the same day as Member 1’s rank and tie-break rule. After the trial, Member 1 wrote the webcam consent text (17 September), and in launch week watched signups, OTP delivery and login errors (22 to 24 September) while I watched locks and snapshots.

7.3 Collaboration with Member 2 (Harsh Sharma)

Member 2 stored submitted code for similarity checks (31 July), wrote the comparison for submissions inside a room (10 and 11 August) and produced the flag list for the host dashboard and admin (12 and 13 August). My dashboard of 17 to 20 August is the host-side place for evidence about players, next to camera status, violation counts and snapshots. After the trial, Member 2 built the review tools for flagged cheating and eliminated players (26 and 27 August), which use the records that my proctoring produces.

False locks and disputes are linked, because a player who is locked wrongly will complain. Member 2 built the dispute form, the payout block and the admin review (17 to 20 August). In the testing weeks Member 2 ran the app as a stranger from signup to match (14 September), at the same time as my mock matches. After launch Member 2 reviewed flagged cheaters (23 September) and the disputes from the first event (24 September).

7.4 Collaboration with Member 3 (Hemanth Singh Tanwar)

Member 3 owns the schema, so my duel and team tables of 17 July were added together with Member 3 after schema V1 (13 July), the migration runner (14 July) and the repository layer (15 July). Rate limits (20 July), input validation (21 July) and the error handler (23 July) of Member 3 sit around my routes. Member 3 turned on backups on 29 July, deployed

Milestone 1 to staging on 30 July and ran the test checklist on 31 July, which was the same time that I tested and fixed the duel invite flow.

In the security review of 31 August to 3 September, Member 3 checked that every route uses parameterized queries and tested injection, session fixation and token misuse; my routes are among those checked. The load simulation for 200 and 500 players (7 and 8 September) and the scoreboard cache (10 September) came in the same week as my edge-case tests. Member 3 created the separate production service and database (14 and 15 September), ran backup restore tests and error alerts (18 September), deployed to production (22 September), ran the first real tournament (23 September) and checked every payout (24 September).

7.5 Collaboration with Member 5 (Akshat Rathore)

Member 5 gave my pages their look. The components, base layout and navigation (14 to 16 July) are what my duel page and team screens use. For tournament mode Member 5 designed the consent screen (10 August), the camera check screen (11 August) and the lock screen (12 August), while I built the camera permission and face check (10 August), the snapshot (11 August) and the locked screen (14 August). The teammate list that I built on 29 July sits inside the match page that Member 5 built from 27 to 30 July.

Member 5 built achievements with metrics from real counts, including duels won (17 to 20 August), and the My mistakes list (5 to 7 August); on 2 and 3 September I fed duel results into both. After launch Member 5 collected feedback from the first players (24 September), reviewed it (28 September) and fixed small UI issues (29 September), the same day on which I tuned the lock rules.

7.6 Team-Wide Activities

All members follow the working rhythm of the plan: a short planning meeting on Monday, a stand-up on Wednesday, a demo on Friday and a retro. The main branch is always deployable, every change merges through a pull request that another member reviews, and shared files such as schema.sql, HackAgonApplication.java and api.js are changed through Member 3 and Member 5. The whole team held a retro on 25 September, I reviewed my teammates' pull requests on 5 October, and every member's log ends with the final review and submission preparation on 6 October.

7.7 Summary of Collaboration

My modules guard the contest while it is running and add formats around it. The lock is useful only because Member 1 enforces it, the evidence is useful only because Member 2 reviews it, my tables and storage are safe only because Member 3 hosts and secures them, and my screens are usable only because Member 5 designed them. Appendix B lists 27 dated joint items. Figure 7.2 counts them for each partner and splits them between the build weeks and the hardening and launch weeks. Member 1 is the busiest partner, which matches the shared lock, consent, elimination and PlayController.java work.

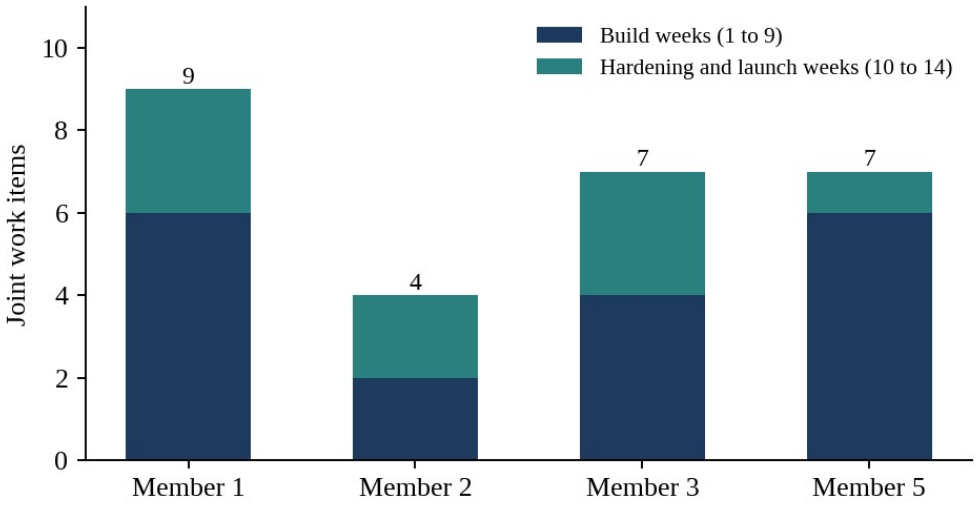


Figure 7.2: Joint work items per partner (from Appendix B)

8. TESTING AND VERIFICATION

8.1 Test Strategy

The plan's definition of done asks for a feature that works in the browser, has a test or smoke check, shows no fake numbers, has hand-written messages and is merged to main. For proctoring, a browser test alone is not enough, because the right answer depends on how real people behave. I therefore used four levels: a small test with each feature, a trial with ten people, tests of unusual situations, and mock matches with friends.

8.2 Tests Recorded in the Work Log

Table 8.1: Tests carried out by Member 4

Date	Test	What it checked
24-07-2026	Team size rules	Duo and squad size limits (tested on the day they were added)
30-07-2026	Duel invite flow	An invitation leads to a started duel
07-08-2026	Duo, squad and duel play	All three formats work with the same room and match page
14-08-2026	Locked screen	The screen appears for a locked player
24-08-2026	10-person proctoring test match	Proctoring with ten real players
25-08-2026	Count of false locks	How often honest players were locked
07-09-2026	Refresh and re-login during a match	State survives a refresh
08-09-2026	Two tabs open	The second tab is handled correctly
09-09-2026	Camera unplugged	A missing camera is detected
10-09-2026	Slow network	Honest players on a weak connection
11-09-2026	Locks survive; honest players are not punished	Final confirmation of the edge cases
14-09 to 16-09-2026	Mock matches 1, 2 and 3 with friends	Whole tournament-mode flow under real use
30-09-2026	Re-test of duels and teams	The formats still work after the first event
02-10-2026	Snapshot deletion job	Old snapshots are removed

8.3 Bug-fix Days

Bug-fix and tuning entries appear on 31 July (bugs fixed), 26 August (grace time tuned), 27 August (problems on different browsers), 4 September (problems from the proctoring trial), 17 September (bugs from the mock matches), 24 September (urgent proctoring problems) and 29 September (lock rules tuned to reduce false locks). The log does not list each bug, so this report does not either.

8.4 Verification by Other Members

Member 3's test checklist before Milestone 1 (31 July) and the security tests (31 August to 3 September) covered code that I wrote. Member 3's load simulation (7 and 8 September)

exercised the match and scoreboard paths that my proctoring script runs on. Member 2's stranger test (14 September), Member 1's end-to-end script (7 to 9 September, re-run on 30 September) and the first real tournament (23 September) exercised the room, join and lock paths under real use.

8.5 Points to Note

- The log records the false-lock count as a task (25 August) but not the number or the findings of the trial (28 August).
- The log records that browser differences were noted (9 July) but not what they were.
- The final grace time and lock rules after tuning are not stated in the log.
- The plan lists host review and reinstatement of a wrongly locked player as a fallback for false locks. My log has no reinstate task, so I do not claim one.

9. WEEK-BY-WEEK SUMMARY

Table 9.1 gives the week-by-week summary. Weeks 1 to 13 have 14.5 hours each and week 14 has 5.5 hours.

Table 9.1: Weekly summary of Member 4's work

Week	Dates (log)	Work	Hours
1	06-07 to 10-07	Browser spike on tab, minimize, fullscreen and camera events; browser differences; proctor API contract	14.5
2	13-07 to 17-07	Snapshot, violation and lock-state endpoints; snapshot storage decision; duel and team tables with Member 3	14.5
3	20-07 to 24-07	Duel invite API, waiting-queue API, create and join team APIs, team size rules	14.5
4	27-07 to 31-07	Duel page connected, team join screen, teammate list; invite flow tested; bugs fixed	14.5
5	03-08 to 07-08	Team score counts each question once; automatic pairing; daily room; practice duels; play test	14.5
6	10-08 to 14-08	Camera permission and face check; snapshots; server lock; 30 second grace; locked screen	14.5
7	17-08 to 21-08	Host dashboard: player list, camera status, violation counts, snapshot viewer, eliminate button; 30 day deletion job	14.5
8	24-08 to 28-08	10-person trial; false locks counted; grace tuned; browser fixes; findings written	14.5
9	31-08 to 04-09	Rematch, duel history, duel results to achievements and mistakes; trial fixes	14.5
10	07-09 to 11-09	Refresh, re-login, two tabs, unplugged camera and slow-network tests; locks confirmed	14.5
11	14-09 to 18-09	Three mock matches; bugs fixed; short host guide	14.5
12	21-09 to 25-09	Monitoring prepared; locks and snapshots monitored; urgent fixes; retro	14.5
13	28-09 to 02-10	Logs reviewed; lock rules tuned; duels and teams re-tested; host guide updated; deletion job checked	14.5
14	05-10 to 06-10	Pull request review; final review and submission preparation	5.5
		Total	**194.0**

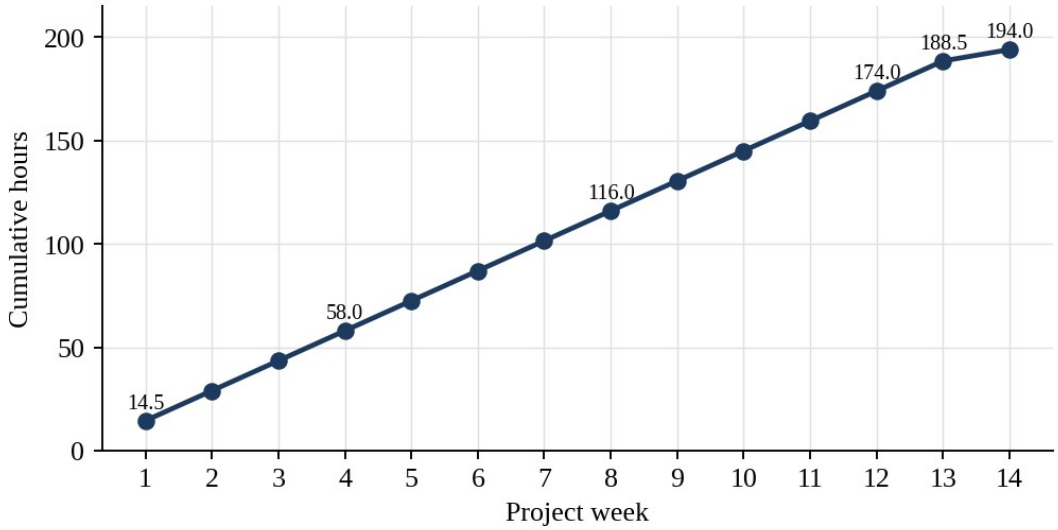


Figure 9.1: Cumulative hours logged by Member 4 week by week

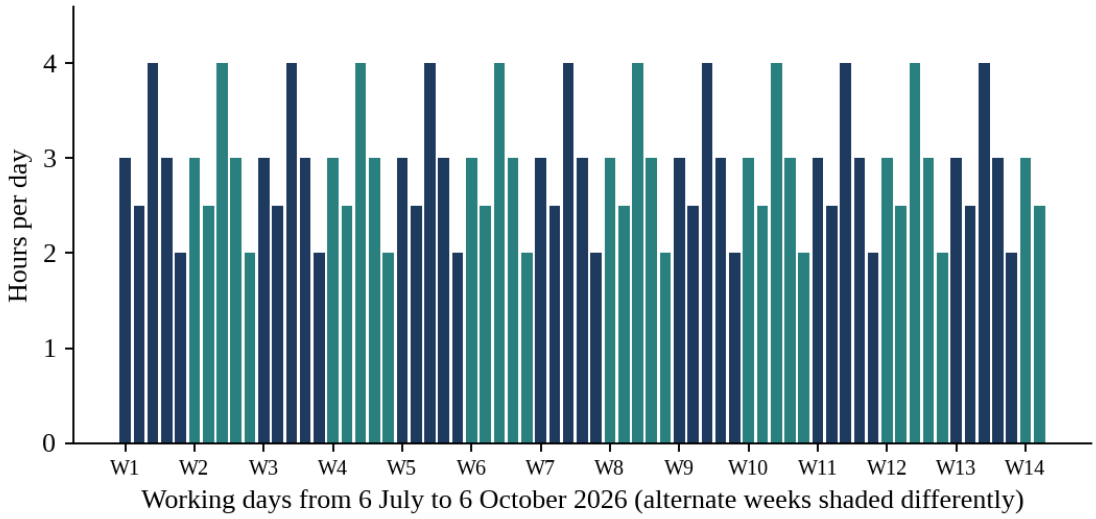


Figure 9.2: Hours logged per day, 6 July to 6 October 2026 (one bar per weekday)

9.1 Weeks 1 and 2: Spike and Endpoints

The first week answered the question whether browsers can report what proctoring needs, and ended with a written contract. The second week turned the contract into three endpoints, chose where snapshots live and added the duel and team tables.

9.2 Weeks 3 to 5: Duels, Teams and Formats

Week 3 delivered the APIs, week 4 the screens and a tested invite flow, and week 5 the rules and automatic parts: team scoring, pairing, the daily room and practice duels. The week ended with a test of duo, squad and duel play.

9.3 Weeks 6 to 9: Lock, Dashboard, Trial and Duel Polish

Week 6 made the lock real, with the face check, snapshots, the server lock, the grace period and the locked screen. Week 7 gave the host a dashboard and a retention job. Week 8 ran the first real trial and tuned the result. Week 9 returned to duels with rematch and history, and fed duel results into achievements and mistakes.

9.4 Weeks 10 to 12: Edge Cases, Mock Matches and Launch

Week 10 tested the awkward situations. Week 11 ran three mock matches and wrote the host guide. Week 12 began with the preparation of monitoring, followed by the first real event.

9.5 Weeks 13 and 14: After Launch and Handover

Week 13 reviewed the logs of the event, tuned the lock rules, re-tested duels and teams, updated the host guide and checked the deletion job. Week 14 reviewed pull requests and prepared the submission.

10. CHALLENGES, DECISIONS AND LIMITATIONS

10.1 Challenges and How They Were Met

Table 10.1: Challenges and responses

Challenge	Response
Browsers report tab, minimize and fullscreen events differently	Spike in Chrome, Edge and Firefox before building; browser differences noted; fixes on 27 August
Other modules needed the proctor API before it was ready	Contract written on 10 July; endpoints built as stubs in week 2
A lock held only in the browser could be cleared by refreshing	Lock state stored and checked on the server (12 August); refresh and re-login tested on 7 September
A short network drop could lock an honest player	30 second grace period (13 August), tuned after the trial (26 August) and again after launch (29 September)
Snapshots are personal images and must not pile up	Snapshot storage chosen on 16 July; deletion after 30 days (21 August); job re-checked on 2 October
A single test page cannot show real behaviour	10-person trial, edge-case tests and three mock matches
A team could inflate its score by repeating a question	Team score counts each question once (3 August)
A new platform has few players	Platform daily room and always-open practice duels (5 and 6 August)
Launch problems cannot be fully predicted	Monitoring prepared on 21 September; locks and snapshots watched on 22 and 23 September; urgent fixes on 24 September

10.2 Delivered Features

Table 10.2: Status of Member 4's features

Feature	Status
Browser spike and proctor API contract	Delivered (log, 6 to 10 July)
Snapshot, violation and lock-state endpoints; storage decision; duel and team tables	Delivered (log, 13 to 17 July)
Duel invite, waiting queue, create and join team, size rules	Delivered (log, 20 to 24 July)
Duel page, team join screen, teammate list; invite flow tested	Delivered (log, 27 to 31 July)
Team scoring, automatic pairing, daily room, practice duels	Delivered (log, 3 to 7 August)
Face check, snapshots, server lock, grace period, locked screen	Delivered (log, 10 to 14 August)
Host dashboard and snapshot deletion job	Delivered (log, 17 to 21 August)
Proctoring trial, tuning and findings	Delivered (log, 24 to 28 August)
Rematch, duel history, duel results into achievements and mistakes	Delivered (log, 31 August to 4 September)

Feature	Status
Edge-case tests and mock matches; host guide	Delivered (log, 7 to 18 September)
Launch monitoring and urgent fixes	Delivered (log, 21 to 25 September)
Post-launch tuning, re-tests, host guide update and report preparation	Delivered (log, 28 September to 6 October)

10.3 Notes on the Sources

- The log describes each task in a line. Details that it does not state, such as the storage used for snapshots, the face-check method, the number of false locks, the final grace time and the findings of the trial, are not invented here.
- Dates and hours are those of the log. They are the only record of what was done on each day.
- The plan lists twelve weeks starting on 12 October 2026, whereas the log covers fourteen weeks from 6 July 2026. I follow the log for dates and the plan for the meaning of tasks.
- The logs of different members place some launch-week events on slightly different days (Section 6.4).

10.4 Limitations

This report relies on the plan and the log and does not contain test output or code listings. Hours are as recorded. Proctoring by browser has natural limits: it can see tab changes, minimizing, fullscreen exits, the camera and the network, but it cannot see a second device or a person sitting next to the player. The lock and the snapshots therefore support the host’s judgement and do not replace it.

11. CONCLUSION AND FUTURE SCOPE

11.1 Conclusion

Over fourteen logged weeks and 194.0 hours I delivered the proctoring and host dashboard module and the duels and teams module of HackAgon, and took both through a trial, edge-case tests, mock matches and the first real event. In tournament mode a player gives consent, passes a camera check, is photographed every 20 to 30 seconds and is locked on the server if the tab or fullscreen rules are broken, with a 30 second grace period for network problems. A host can follow the players on a dashboard, view snapshots and eliminate a player, and snapshots are deleted after 30 days. Players can also duel by invite, queue or automatic pairing, play in duo and squad teams where a question counts once, use the daily room and practice duels, rematch, and see duel results feed their achievements and mistakes. Each task is traced to a dated log entry and to the partner module it connects with.

11.2 Lessons Learnt

- A spike before building saves weeks, because it shows how browsers really behave.
- A written contract lets three members work in parallel on one lock.
- A lock that is stored on the server is the base of any fair check, and the browser should only report.
- A grace period and a trial matter as much as the lock, because wrongly locked honest players cost more trust than a few missed cheaters.
- Tests with real people, such as the 10-person trial and the mock matches, found what single-person tests could not.
- Data that is personal, such as snapshots, needs a retention rule from the start.

11.3 Future Scope

- Record the number of false locks of every event and show it on the admin dashboard.
- Add the host review and reinstatement of a wrongly locked player that the plan lists as a fallback, if it is not yet in place.
- Repeat the trial with a larger group and on more devices, including mobile browsers.
- Show the similarity flags of Member 2 on the host dashboard next to camera status and snapshots.
- Have the consent text, the snapshot retention period and the storage reviewed by a legal adviser before wider use.
- Add rating or ladders to duels and more team sizes once duels are used often.

REFERENCES

- HackAgon Team Modules and 12-Week Plan, HackAgon team, 2026.
- HackAgon Daily Work Log, 6 July 2026 to 6 October 2026, HackAgon team.
- Java SE 17 documentation, docs.oracle.com.
- Spring Boot reference documentation, docs.spring.io.
- MySQL Reference Manual, dev.mysql.com.
- MDN Web Docs, pages on page visibility, fullscreen and camera capture in the browser, developer.mozilla.org.

APPENDIX A: COMPLETE DAILY WORK LOG OF MEMBER 4

The log below is reproduced from the HackAgon Daily Work Log, Member 4 (Proctoring, Duels), 6 July 2026 to 6 October 2026. Total: 67 days, 194.0 hours.

Table A.1: Daily work log of Member 4

Date	Hrs	Status	Work description
Week 1			
06-07-2026	3	Completed	Tested tab-change event in Chrome
07-07-2026	2.5	Completed	Tested minimize and fullscreen-exit events
08-07-2026	4	Completed	Tested camera capture in Edge and Firefox
09-07-2026	3	Completed	Noted browser differences
10-07-2026	2	Completed	Wrote proctor API contract for snapshot, violation, lock state
Week 2			
13-07-2026	3	Completed	Built snapshot upload endpoint
14-07-2026	2.5	Completed	Built violation log endpoint
15-07-2026	4	Completed	Built lock-state endpoint
16-07-2026	3	Completed	Chose where snapshots are stored
17-07-2026	2	Completed	Added duel and team tables with Member 3
Week 3			
20-07-2026	3	Completed	Built duel invite API
21-07-2026	2.5	Completed	Built duel waiting-queue API
22-07-2026	4	Completed	Built create team API
23-07-2026	3	Completed	Built join team API for duo and squad
24-07-2026	2	Completed	Added team size rules and tested
Week 4			
27-07-2026	3	Completed	Connected duel.html to the duel API
28-07-2026	2.5	Completed	Built team join screen
29-07-2026	4	Completed	Built teammate list inside a room
30-07-2026	3	Completed	Tested duel invite flow
31-07-2026	2	Completed	Fixed bugs
Week 5			
03-08-2026	3	Completed	Made team score count each question once
04-08-2026	2.5	Completed	Built automatic duel pairing
05-08-2026	4	Completed	Built platform daily room

Date	Hrs	Status	Work description
06-08-2026	3	Completed	Built always-open practice duels
07-08-2026	2	Completed	Tested duo, squad and duel play
Week 6			
10-08-2026	3	Completed	Added camera permission and face check
11-08-2026	2.5	Completed	Added snapshot every 20 to 30 seconds
12-08-2026	4	Completed	Set tab and minimize lock on server
13-08-2026	3	Completed	Added 30 second network grace period
14-08-2026	2	Completed	Built locked screen and tested
Week 7			
17-08-2026	3	Completed	Built host dashboard live player list
18-08-2026	2.5	Completed	Added camera status and violation counts
19-08-2026	4	Completed	Built snapshot viewer
20-08-2026	3	Completed	Added eliminate button
21-08-2026	2	Completed	Built job that deletes snapshots after 30 days
Week 8			
24-08-2026	3	Completed	Ran 10-person proctoring test match
25-08-2026	2.5	Completed	Counted false locks
26-08-2026	4	Completed	Tuned grace time
27-08-2026	3	Completed	Fixed problems on different browsers
28-08-2026	2	Completed	Wrote findings
Week 9			
31-08-2026	3	Completed	Built rematch for duels
01-09-2026	2.5	Completed	Built duel history
02-09-2026	4	Completed	Fed duel results to achievements
03-09-2026	3	Completed	Fed duel results to mistakes
04-09-2026	2	Completed	Fixed problems from proctoring trial
Week 10			
07-09-2026	3	Completed	Tested refresh and re-login during a match
08-09-2026	2.5	Completed	Tested two tabs open
09-09-2026	4	Completed	Tested camera unplugged
10-09-2026	3	Completed	Tested slow network
11-09-2026	2	Completed	Confirmed locks survive and honest players are not punished

Date	Hrs	Status	Work description
Week 11			
14-09-2026	3	Completed	Ran mock match 1 with friends
15-09-2026	2.5	Completed	Ran mock match 2 with friends
16-09-2026	4	Completed	Ran mock match 3 with friends
17-09-2026	3	Completed	Fixed bugs found in mock matches
18-09-2026	2	Completed	Wrote short host guide for tournament mode
Week 12			
21-09-2026	3	Completed	Prepared monitoring for the event
22-09-2026	2.5	Completed	Monitored locks during the event
23-09-2026	4	Completed	Monitored snapshots during the event
24-09-2026	3	Completed	Fixed urgent proctoring problems
25-09-2026	2	Completed	Held team retro
Week 13			
28-09-2026	3	Completed	Reviewed proctoring logs from the event
29-09-2026	2.5	Completed	Tuned lock rules to reduce false locks
30-09-2026	4	Completed	Re-tested duels and teams
01-10-2026	3	Completed	Updated host guide for tournament mode
02-10-2026	2	Completed	Checked snapshot deletion job
Week 14			
05-10-2026	3	Completed	Reviewed teammates' pull requests
06-10-2026	2.5	Completed	Final review and submission preparation

APPENDIX B: DATED JOINT WORK WITH OTHER MEMBERS

Each row pairs a task of Member 4 with the task of another member that it depends on or supports, as recorded in the daily log.

Table B.1: Dated joint work

Member 4 task	Other member task	With
10-07: proctor API contract written (snapshot, violation, lock state)	01-10: api-contract.md updated for accounts and rooms, with the proctor contract as a reference	Member 1
12-08 and 13-08: tab and minimize lock on the server; 30 second grace period	12-08: match page refuses locked players; 13-08: lock check tested on every request	Member 1
10-08 and 11-08: camera permission, face check and snapshots	11-08: consent page built before joining	Member 1
10-08 and 11-08: camera permission, face check and snapshots	10-08: consent screen designed; 11-08: camera check screen designed	Member 5
14-08: locked screen built and tested	12-08: lock screen designed	Member 5
17-07: duel and team tables added	13-07: schema V1 tables fixed; 14-07: migration runner; 15-07: repo layer	Member 3
20-07 to 24-07: duel and team APIs	20-07: rate limits on login and OTP; 21-07: input validation; 23-07: error handler and request logs	Member 3
27-07 and 28-07: duel.html connected; team join screen	14-07 to 16-07: components, base layout and shared navigation	Member 5
29-07: teammate list inside a room	27-07 to 30-07: match.html with editor, timer, polling scoreboard and countdown	Member 5
29-07: teammate list inside a room	27-07 and 28-07: join by Arena and invite code; start match logic	Member 1
03-08: team score counts each question once	03-08: final ranks and tie-break rule; 04-08: results page data	Member 1
30-07 and 31-07: duel invite flow tested; bugs fixed	30-07: Milestone 1 deployed to staging; 31-07: test checklist run	Member 3
02-09: duel results fed to achievements	17-08 to 20-08: achievements with metrics from real counts, including duels won	Member 5
03-09: duel results fed to mistakes	03-08: hook in PlayController.java agreed; 05-08 to 07-08: My mistakes list	Member 5
17-08 to 19-08: host dashboard with player list, camera status, violation counts and snapshot viewer	12-08: flag list produced for the host dashboard; 13-08: flag list shown in admin	Member 2
20-08: eliminate button on the host dashboard	17-08: host eliminates a player with a reason; 18-08: rejoin blocked and score removed from ranking	Member 1

Member 4 task	**Other member task**	**With**
24-08 to 28-08: proctoring trial and findings	26-08: review tool for flagged cheating; 27-08: review tool for eliminated players	Member 2
16-07 and 21-08: snapshot storage chosen; 30 day deletion job	29-07: backups on; 14-09 and 15-09: separate production service and database	Member 3
07-09 to 11-09: refresh, two-tab, camera and slow-network tests	07-09 and 08-09: load simulation up to 500 players; 10-09: short scoreboard cache	Member 3
14-09 to 16-09: mock matches with friends	14-09: app tested as a stranger from signup to match; 15-09 to 17-09: bugs logged and fixed	Member 2
14-09 to 18-09: mock matches and host guide	17-09: webcam consent text written; 18-09: acceptance checkbox added at signup	Member 1
21-09 to 23-09: monitoring of locks and snapshots	22-09: deploy to production; 23-09: first real tournament; 24-09: payouts checked	Member 3
22-09 to 24-09: monitoring and urgent proctoring fixes	22-09 to 24-09: signups, OTP delivery, login errors and the first tournament watched	Member 1
23-09 and 24-09: flagged cheaters and disputes	23-09: flagged cheaters reviewed; 24-09: disputes from the first event reviewed	Member 2
29-09: lock rules tuned to reduce false locks	24-09 and 28-09: feedback from first players collected and reviewed; 29-09: small UI issues fixed	Member 5
30-09: duels and teams re-tested	30-09: end-to-end test script re-run	Member 1
02-10: snapshot deletion job checked	01-10: backups verified to restore; 02-10: deployment notes updated	Member 3